

USB Host Function

Experiment Objective

In this experiment, you will learn how to implement the USB host function on the MicroROS-V2 control board, connecting a USB keyboard, mouse, or gamepad through the ESP32-S3's USB OTG interface. Through this example, you will master the use of the USB Host Library and the HID (Human Interface Device) class driver in ESP-IDF.

Experiment Principle

1. Keywords

Keyword	Description
USB OTG	USB On-The-Go, allows a device to switch between host and device modes; the ESP32-S3 supports OTG
HID	Human Interface Device, including keyboards, mice, gamepads, etc.
Boot protocol	A standard protocol mode of HID devices that reports keyboard/mouse input data in a fixed format
Report descriptor	A structure that describes the data format of a HID device, defining the meaning and length of each field
Endpoint	A logical channel of USB communication; HID devices usually use interrupt endpoints to transfer data
USB Host Library	The USB host stack provided by ESP-IDF, responsible for device enumeration, transfer management, etc.

2. Technical Architecture

The program adopts a multitasking architecture: the USB library task handles USB bus events, the HID host task handles HID device events, and the application entry is responsible for initialization and startup. When a USB HID device is connected, the program automatically identifies the device type (keyboard/mouse/gamepad) and parses the data of the corresponding protocol. The keyboard and mouse are parsed using the Boot protocol, while the gamepad is parsed through a custom 9-byte report format. Key files are located in `~/esp/base_samples/usb_host/hid/`, including `main/hid_host_example.c` (main program entry), `main/joydata_gamepad.h` (gamepad data structure) and `main/joydata_gamepad.c` (gamepad data parsing), `main/CMakeLists.txt` (build configuration) and `main/idf_component.yml` (component dependencies).

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB Type-C data cable	✓ Required
USB keyboard/mouse/gamepad	✓ Required (choose any one)
USB OTG adapter cable (Type-A female)	✓ Required

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Connect the USB HID device to the control board using the USB OTG adapter cable

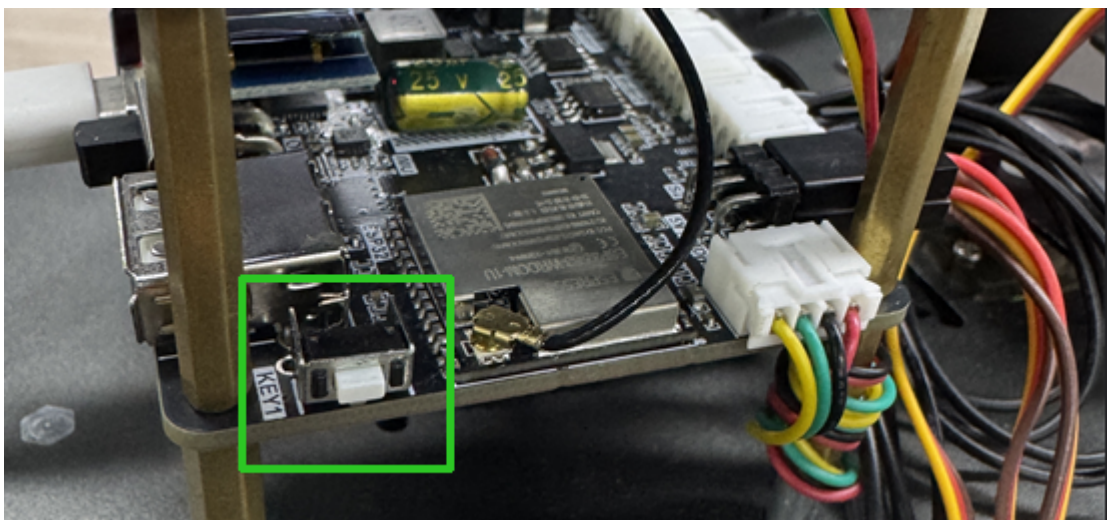
3. Compile and Flash

```
cd ~/esp/base_samples/usb_host/hid
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

Note: This example depends on the `espressif/usb_host_hid` component (version 1.0.0), which is automatically downloaded from the Espressif Component Registry on the first compilation.

4. How to Stop

Press the **Boot button (GPIO0)** on the control board to exit the program, or press `Ctrl+]` in the terminal to exit the serial monitor.



Experiment Result

After compiling and flashing successfully, when a keyboard is connected, the serial output is:

```
I (xxx) example: HID Host example
I (xxx) example: HID Device, protocol 'KEYBOARD' CONNECTED
Keyboard
Hello, ESP32 USB HID keyboard is here!
```

When a mouse is connected, the output is:

```
Mouse
X: 000883      Y: 000058      |o| |
```

When a gamepad is connected, the output is:

```
JOYDATA LJ(h= 1 v= 0 btn=0) RJ(h= 0 v= 0 btn=0) DPAD(h= 0 v= 0) A=0 B=0 X=0 Y=0 L1=0 L2=0
R1=0 R2=0 SEL=0 START=0 ts=12345 ms
```

Press the **Boot button (GPIO0)** on the control board to exit the program.

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/usb_host/hid/main/hid_host_example.c`
- Gamepad data: `~/esp/base_samples/usb_host/hid/main/joydata_gamepad.h`, `joydata_gamepad.c`

1. Main Program Logic

`main/hid_host_example.c` is the program entry, and the overall architecture is divided into three tasks:

1. USB library task (`usb_lib_task`):

- Configure GPIO0 as input (Boot button, press to exit)
- Call `usb_host_install()` to install the USB host library
- Loop calling `usb_host_lib_handle_events()` to handle USB events
- When GPIO0 becomes low, call `usb_host_uninstall()` to uninstall the USB library

2. HID host task (`hid_host_task`):

- Create the event queue `hid_host_event_queue`
- Receive events from the HID driver and call `hid_host_device_event()` to handle them

3. Application entry (`app_main`):

- Create the USB library task and wait for its initialization to complete
- Call `hid_host_install()` to install the HID host driver (with a built-in background task)
- Create the HID host task to handle device events

2. Key Components

USB hardware pins (see the README):

ESP32-S3	USB Type-A connector
GPI019	----> D-
GPI020	----> D+

Exit button: GPIO0 (Boot button), press to exit the program.

main/hid_host_example.c — main program file, core functions:

Function	Description
app_main()	Program entry, initializes the USB host library and HID driver
usb_lib_task()	USB library background task, handles USB bus events
hid_host_task()	HID event handling task, receives and processes device events from the queue
hid_host_device_event()	Handles HID device connection events, opens the device and starts data reception
hid_host_interface_callback()	HID interface event callback, handles input reports, disconnection, etc.
hid_host_keyboard_report_callback()	Handles keyboard HID reports, converting key codes to ASCII characters
hid_host_mouse_report_callback()	Handles mouse HID reports, computing and outputting X/Y coordinates and button states
hid_host_generic_report_callback()	Handles generic HID reports (gamepad), parsing button and joystick data

HID driver configuration:

Parameter definition file: ~/esp/base_samples/usb_host/hid/main/hid_host_example.c

```
const hid_host_driver_config_t hid_host_driver_config = {
    .create_background_task = true,    // Create the background event handling task
    .task_priority = 5,
    .stack_size = 4096,
    .core_id = 0,
    .callback = hid_host_device_callback,
    .callback_arg = NULL
};
```

Device connection handling:

When a HID device is connected, the `HID_HOST_DRIVER_EVENT_CONNECTED` event in `hid_host_device_event()` is triggered, and the following operations are performed:

1. Call `hid_host_device_open()` to open the device
2. For Boot subclass devices, call `hid_class_request_set_protocol()` to set the Boot protocol
3. For keyboard devices, call `hid_class_request_set_idle()` to set the idle rate
4. Call `hid_host_device_start()` to start receiving data

3. Keyboard Data Processing

The keyboard uses the Boot protocol and is parsed through the `hid_keyboard_input_report_boot_t` structure. `hid_host_keyboard_report_callback()` maintains an array of the previous frame's key states, detects press and release events by comparing the old and new key arrays, and converts the key code to an ASCII character for output via `hid_keyboard_get_char()`.

4. Mouse Data Processing

The mouse uses the Boot protocol and is parsed through the `hid_mouse_input_report_boot_t` structure. `hid_host_mouse_report_callback()` accumulates the absolute X/Y position and displays the left/right button states of the mouse.

5. Gamepad Data Processing

`main/joydata_gamepad.h` — gamepad data structure definitions:

```
typedef struct {
    int8_t left_joystick_h;           // Left joystick horizontal (left=+1, right=-1,
center=0)
    int8_t left_joystick_v;           // Left joystick vertical (up=+1, down=-1, center=0)
    uint8_t left_joystick_button;      // Left joystick button
    int8_t right_joystick_h;           // Right joystick horizontal
    int8_t right_joystick_v;           // Right joystick vertical
    uint8_t right_joystick_button;     // Right joystick button
    int8_t left_button_upper_plane_h;  // D-pad horizontal
    int8_t left_button_upper_plane_v;  // D-pad vertical
    uint8_t right_button_upper_plane_a, b, x, y; // ABXY buttons
    uint8_t left_button_frontal_plane_l1, l2; // L1/L2 buttons
    uint8_t right_button_frontal_plane_r1, r2; // R1/R2 buttons
    uint8_t select, start;
    uint32_t timestamp_ms;             // Timestamp
} joy_data_t;
```

`main/joydata_gamepad.c` — gamepad data parsing (9-byte report format):

Byte	Content
data[0]	A(bit0), B(bit1), X(bit3), Y(bit4), L1(bit6), R1(bit7)
data[1]	L2(bit0), R2(bit1), SELECT(bit2), START(bit3), L3(bit5), R3(bit6)

Byte	Content
data[2]	D-pad HAT value (0x0F neutral, 0x00~0x07 correspond to 8 directions)
data[3]	Left joystick X-axis raw value (0x80 centered, dead zone ± 20)
data[4]	Left joystick Y-axis raw value
data[5]	Right joystick X-axis raw value
data[6]	Right joystick Y-axis raw value
data[7]	R2 analog value (triggered when the threshold > 0x20)
data[8]	L2 analog value (triggered when the threshold > 0x20)

main/CMakeLists.txt — component registration:

```
idf_component_register(  
    SRCS  
        "hid_host_example.c"  
        "joydata_gamepad.c"  
    INCLUDE_DIRS "."  
    PRIV_REQUIRES driver usb  
)
```

main/idf_component.yml — component dependency declarations:

```
dependencies:  
  idf: ">=4.4"  
  usb_host_hid: "1.0.0"
```

Common Problems

Problem	Cause	Solution
USB device not recognized	Wrong USB D+/D- pin connections	Confirm GPIO19 (D-) and GPIO20 (D+) are connected correctly
Compilation fails, cannot find the <code>usb_host_hid</code> component	Missing component dependency or network issues	Check the network connection and make sure the Espressif Component Registry is accessible
Keyboard has no output	The keyboard does not support the Boot protocol	Try a standard USB keyboard instead
The program cannot exit	GPIO0 (Boot button) not connected correctly	Press the Boot button on the control board briefly to pull GPIO0 low

Problem	Cause	Solution
Garbled gamepad data	The gamepad report format is incompatible with this example	Different brands of gamepads may have different report formats; adjust the parsing logic according to the actual data